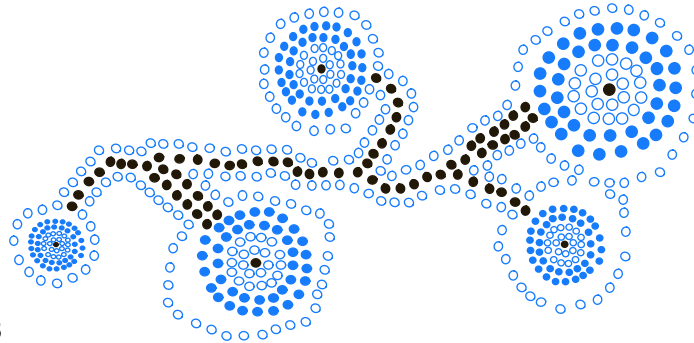




Contents

9.1	The Priority Queue Abstract Data Type	360
9.1.1	Priorities	360
9.1.2	The Priority Queue ADT	361
9.2	Implementing a Priority Queue	362
9.2.1	The Entry Composite	362
9.2.2	Comparing Keys with Total Orders	363
9.2.3	The AbstractPriorityQueue Base Class	364
9.2.4	Implementing a Priority Queue with an Unsorted List . . .	366
9.2.5	Implementing a Priority Queue with a Sorted List	368
9.3	Heaps	370
9.3.1	The Heap Data Structure	370
9.3.2	Implementing a Priority Queue with a Heap	372
9.3.3	Analysis of a Heap-Based Priority Queue	379
9.3.4	Bottom-Up Heap Construction ★	380
9.3.5	Using the java.util.PriorityQueue Class	384
9.4	Sorting with a Priority Queue	385
9.4.1	Selection-Sort and Insertion-Sort	386
9.4.2	Heap-Sort	388
9.5	Adaptable Priority Queues	390
9.5.1	Location-Aware Entries	391
9.5.2	Implementing an Adaptable Priority Queue	392
9.6	Exercises	395

9.1 The Priority Queue Abstract Data Type

9.1.1 Priorities

In Chapter 6, we introduced the queue ADT as a collection of objects that are added and removed according to the *first-in, first-out (FIFO)* principle. A company's customer call center embodies such a model in which waiting customers are told "calls will be answered in the order that they were received." In that setting, a new call is added to the back of the queue, and each time a customer service representative becomes available, he or she is connected with the call that is removed from the front of the call queue.

In practice, there are many applications in which a queue-like structure is used to manage objects that must be processed in some way, but for which the first-in, first-out policy does not suffice. Consider, for example, an air-traffic control center that has to decide which flight to clear for landing from among many approaching the airport. This choice may be influenced by factors such as each plane's distance from the runway, time spent waiting in a holding pattern, or amount of remaining fuel. It is unlikely that the landing decisions are based purely on a FIFO policy.

There are other situations in which a "first come, first serve" policy might seem reasonable, yet for which other priorities come into play. To use another airline analogy, suppose a certain flight is fully booked an hour prior to departure. Because of the possibility of cancellations, the airline maintains a queue of standby passengers hoping to get a seat. Although the priority of a standby passenger is influenced by the check-in time of that passenger, other considerations include the fare paid and frequent-flyer status. So it may be that an available seat is given to a passenger who has arrived *later* than another, if such a passenger is assigned a better priority by the airline agent.

In this chapter, we introduce a new abstract data type known as a *priority queue*. This is a collection of prioritized elements that allows arbitrary element insertion, and allows the removal of the element that has first priority. When an element is added to a priority queue, the user designates its priority by providing an associated *key*. The element with the *minimal* key will be the next to be removed from the queue (thus, an element with key 1 will be given priority over an element with key 2). Although it is quite common for priorities to be expressed numerically, any Java object may be used as a key, as long as there exists means to compare any two instances *a* and *b*, in a way that defines a natural order of the keys. With such generality, applications may develop their own notion of priority for each element. For example, different financial analysts may assign different ratings (i.e., priorities) to a particular asset, such as a share of stock.

9.1.2 The Priority Queue ADT

We model an element and its priority as a key-value composite known as an *entry*. (However, we defer until Section 9.2.1 the technical definition of the Entry type.)

We define the priority queue ADT to support the following methods:

`insert(k, v)`: Creates an entry with key *k* and value *v* in the priority queue.

`min()`: Returns (but does not remove) a priority queue entry (*k*,*v*) having minimal key; returns null if the priority queue is empty.

`removeMin()`: Removes and returns an entry (*k*,*v*) having minimal key from the priority queue; returns null if the priority queue is empty.

`size()`: Returns the number of entries in the priority queue.

`isEmpty()`: Returns a boolean indicating whether the priority queue is empty.

A priority queue may have multiple entries with equivalent keys, in which case methods `min` and `removeMin` may report an arbitrary choice among those entry having minimal key. Values may be any type of object.

In our initial model for a priority queue, we assume that an element's key remains fixed once it has been added to a priority queue. In Section 9.5, we consider an extension that allows a user to update an element's key within the priority queue.

Example 9.1: *The following table shows a series of operations and their effects on an initially empty priority queue. The “Priority Queue Contents” column is somewhat deceiving since it shows the entries sorted by key. Such an internal representation is not required of a priority queue.*

Method	Return Value	Priority Queue Contents
<code>insert(5,A)</code>		{ (5,A) }
<code>insert(9,C)</code>		{ (5,A), (9,C) }
<code>insert(3,B)</code>		{ (3,B), (5,A), (9,C) }
<code>min()</code>	(3,B)	{ (3,B), (5,A), (9,C) }
<code>removeMin()</code>	(3,B)	{ (5,A), (9,C) }
<code>insert(7,D)</code>		{ (5,A), (7,D), (9,C) }
<code>removeMin()</code>	(5,A)	{ (7,D), (9,C) }
<code>removeMin()</code>	(7,D)	{ (9,C) }
<code>removeMin()</code>	(9,C)	{ }
<code>removeMin()</code>	null	{ }
<code>isEmpty()</code>	true	{ }